



Fullstack Web Development

Session 9 | Week 10/Day 2 | 120 min

Laravel - Part04

Instructor: Ms. Liev Nova



090667393



Outline

- **Passing Data to Views**
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Passing Data to Views

- Laravel **views** can receive data from routes or controllers.
 - The data is usually sent as an **array** of **key-value pairs**.
 - Example using an array:

```
return view('contacts.index', [  
    'data' => $data  
]);
```

- In the view, the variable name becomes:
 - The route can prepare the data.
 - The view can display the data.
 - The code becomes easier to organize.

`$data`

Passing Data to Views

- Laravel gives several ways to pass data into a view.

- **Method 1:** Using an **array**

```
return view('contacts.index', [  
    'data' => $data  
]);
```

- Use this when you want the variable name to be very clear.

- **Method 2:** Using **compact()**

```
return view('contacts.index', compact('data'));
```

- Use this when the **variable name** in **PHP** and the **variable name** in **Blade** are the same.

Passing Data to Views

- Laravel gives several ways to pass data into a view.

- **Method 3:** Using `with()`

```
return view('contacts.index')->with('data', $data);
```

- Use this when you want a fluent chain style.
- *Use the method that makes your code easiest to read.*

Passing Data to Views

- **Creating Sample Contact Data**

- Before using a database, we can store sample data in a service class.

- **Create this file:** `app/Services/Data.php`

- Example:

```
namespace App\Services;

class Data
{
    public static $contacts_data = [
        1 => [
            'name' => 'Sok Dara',
            'gender' => 'Male',
            'email' => 'sok.dara@rupp.edu.kh',
            'phone' => '010 123 123',
            'company' => 'Company 1'
        ],
        2 => [
            'name' => 'Sok Pisey',
            'gender' => 'Female',
            'email' => 'sok.pisey@rupp.edu.kh',
            'phone' => '011 123 123',
            'company' => 'Company 2'
        ],
    ];
}
```

- Why use a separate class?

- The route file stays shorter.
- The data is easier to reuse.
- Later, this data can be replaced with a database model.

Passing Data to Views

- **Displaying Data in Blade**

- The contact list route gets all contact data and sends it to a view.

- Example:

```
use App\Services\Data;  
  
Route::get('/contacts', function () {  
    $data = Data::$contacts_data;  
  
    return view('contacts.index', compact('data'));  
});
```

- *The route controls what data is sent.
The view controls how the data looks.*

- When user visits /contacts.
- Laravel runs the route function.
- The route gets contact data from **Data::\$contacts_data**.
- The route sends the data to contacts.index.
- The Blade view displays the contact list.

Passing Data to Views

- Display Contact List

- Inside the Blade file, use @foreach to loop through the contacts.

```
@foreach ($data as $id => $contact)
    <tr>
        <td>{{ $contact['name'] }}</td>
        <td>{{ $contact['gender'] }}</td>
        <td>{{ $contact['email'] }}</td>
        <td>{{ $contact['phone'] }}</td>
        <td>
            <a href="{{ route('contacts.show', $id) }}">
                Detail
            </a>
        </td>
    </tr>
@endforeach
```

- **\$data** contains all contacts.
- **\$id** is the key of each contact.
- **\$contact** contains one contact record.
- **route('contacts.show', \$id)** creates a link to the detail page.

Passing Data to Views

- **Display Contact List**

- Inside the Blade file, use @foreach to loop through the contacts.

Contact List					
#	Name	Gender	Email	Phone	Detail
1	Sok Dara	Male	sok.dara@rupp.edu.kh	010 123 123	Detail
2	Sok Pisey	Female	sok.pisey@rupp.edu.kh	011 123 123	Detail
3	Sros Bopha	Female	sros.bopha@rupp.edu.kh	090 909 123	Detail
4	Tith Sovan	Male	tith.sovan@rupp.edu.kh	090 902 283	Detail
5	Chan Leakhena	Female	chan.leakhena@rupp.edu.kh	090 233 322	Detail

- **Result:** The **route** sends **contact data** to the **Blade view**, and the **view** displays each contact in a table.

Passing Data to Views

- **Route for Contact Detail**

- A detail page shows information about one contact.

```
Route::get('/contacts/{id}', function ($id) {  
    $data = Data::$contacts_data;  
    $contact = $data[$id] ?? null;  
  
    abort_unless($contact, 404, 'Contact not found');  
  
    return view('contacts.show', compact('contact'));  
})->name('contacts.show');
```

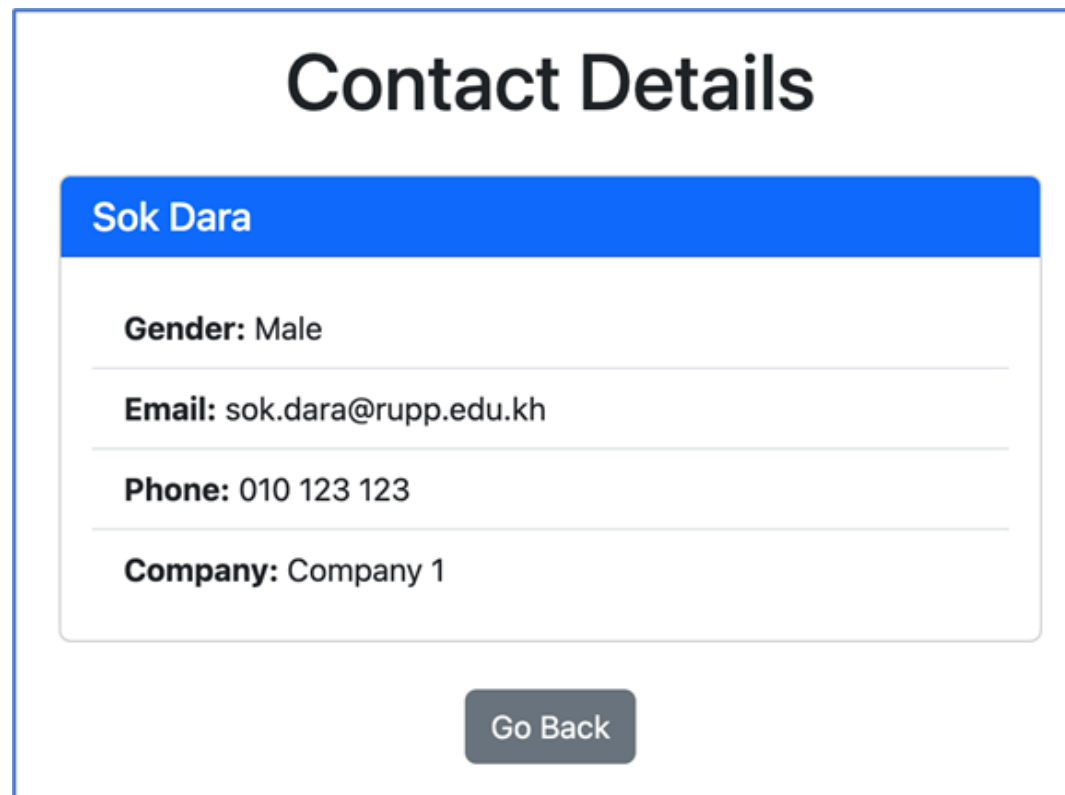
- **{id}** is a route parameter.
- Laravel passes the value into the **\$id variable**.
- The route finds the contact by **id**.
- If the contact does not exist, Laravel returns a **404 page**.
- If the contact exists, Laravel sends it to **contacts.show**.

Passing Data to Views

- **Route for Contact Detail**

- A detail page shows information about one contact.

- **Example Result:**



The screenshot shows a web page titled "Contact Details". At the top, there is a blue header bar with the name "Sok Dara" in white text. Below this, the contact information is displayed in a light gray box with horizontal lines separating the fields: "Gender: Male", "Email: sok.dara@rupp.edu.kh", "Phone: 010 123 123", and "Company: Company 1". At the bottom of the page, there is a dark gray button with the text "Go Back" in white.

- **Important:**

- `{id}` is a route parameter.
- The route uses the `id` to find one contact.
- If the contact does not exist, Laravel returns a 404 page.

Passing Data to Views

- **Handling Missing Records**

- Sometimes the user may request an id that does not exist.

- Example:

`/contacts/99`

- If **contact 99** does not exist, the page should **not break**.

- Laravel provides helper methods for stopping a request when a condition fails.

- `abort_if()`
- `abort_unless()`

Another Approach Passing Data to Views

- The `abort_if` is a method that renders an exception handler if the condition you passed fails.

```
abort_if(!$contact, 404, 'Contact not found');
```

- The `abort_unless()` throws an HTTP exception if a given boolean expression evaluates to false.

```
abort_unless($contact, 404, 'Contact not found');
```

- **Difference:**

- `abort_if` stops the request if the condition is true.
- `abort_unless` stops the request if the condition is false.

- *Use them to make error handling short and readable.*

Another Approach Passing Data to Views

- Laravel provides helper methods for stopping a request when a condition fails.
 - Example using:

```
Route::get( uri: '/contacts/{id}', function($id){  
    $contactsData = Data::$contacts_data;  
    // Abort with a 404 error if the ID does not exist in the data  
    abort_if(!isset($contactsData[$id]), code: 404, message: 'Contact not found.');
```

// Abort with a 404 error unless the ID exists in the data

```
    //abort_unless(isset($contactsData[$id]), 404, 'Contact not found.');
```

\$data = \$contactsData[\$id];

```
    return view( view: 'contacts.show', compact( var_name: 'data' ));  
})->name( name: 'contacts.show');
```

Outline

- Passing Data to Views
- **Building Application Layout**
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Building Application Layout

- An application layout stores the shared **HTML structure** of the website.
- A layout is a shared page structure used by many views.
 - Create file: `resources/views/layouts/main.blade.php`
 - The layout usually contains:
 - HTML structure
 - CSS and JavaScript links
 - Navbar
 - Footer
 - A placeholder for page content
 - Example:
 - When the navbar or layout changes, you only update one file.

```
<body>
    @include('layouts.navbar')

    <main class="container">
        @yield('content')
    </main>
</body>
```


Outline

- Passing Data to Views
- Building Application Layout
- **Blade Template Inheritance**
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Blade Template Inheritance

- **Blade inheritance** lets child views reuse a parent layout.
 - **Parent layout:** `@yield`
 - **Child view:** `@section` and `@extends`

```
<title>@yield('title')</title>

<main>
    @yield('content')
</main>
```

```
@extends('layouts.main')

@section('title', 'Contact List')

@section('content')
    <h1>Contact List</h1>
@endsection
```

- `@extends` connects the child view to the parent layout.
- `@section` defines content for a named area.
- `@yield` displays that content inside the layout.

Blade Template Inheritance

- @yield, @section and @extends

- @yield — Creates a placeholder in the parent layout.

```
@yield('content')
```

- @section — Provides content for a placeholder.

```
@section('content')  
    <h1>Contact List</h1>  
@endsection
```

- @extends — Tells Blade which layout the current view should use.

```
@extends('layouts.main')
```

- *The layout provides the structure. The child view provides the content.*

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- **Including Sub-views**
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Including Sub-views

- The `@include` directive allows one Blade file to include another Blade file.

- Example:

```
<div>  
    @include('contacts._form')  
</div>
```

- This is useful when the same HTML is used in more than one page.

- Examples:

- Form partials
- Alert messages
- Table rows
- Navbar sections
- Reusable buttons

**You write the code once and reuse it in many views.*

Including Sub-views

- **Passing Data to Included Views**

- An included view can use variables from the parent view.
- You can also pass extra data directly.

```
@include('contacts._status', [  
  'status' => 'complete'  
)
```

- Inside `contacts._status`, you can use:

```
{{ $status }}
```

- *The partial becomes more flexible.*
- *The same partial can display different content.*
- *The parent view controls what data is sent.*

Including Sub-views

- **Conditional Include**

- Laravel provides conditional include directives.

- **@includeIf**:

```
@includeIf('contacts._filters')
```

- *Includes the view only if the file exists.*

- **@includeWhen**:

```
@includeWhen($showForm, 'contacts._form')
```

- *Includes the view only when the condition is true.*

- **@includeUnless**:

```
@includeUnless($readonly, 'contacts._actions')
```

- *Includes the view unless the condition is true.*

- Use these when a **partial** depends on a **condition**.

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- **Blade control statements and loops**
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Blade control statements and loops

- **Blade If Statements**

- We can construct if statements using the `@if`, `@elseif`, `@else`, and `@endif` directives. These directives function identically to PHP.
- Use `@if` when:
 - You need to show different content.
 - You need to check a condition.
 - You need to display a message based on data.
- Use **Blade conditions** for display decisions, not complex business logic.

```
@if (count($records) === 1)
    I have one record!
}elseif (count($records) > 1)
    I have multiple records!
@else
    I do not have any records!
@endif
```

Blade control statements and loops

- Blade If Statements

- **@isset** and **@empty**

- **@isset** checks whether a variable **set** and is **not null**.

```
@isset($records)
    Records are available.
@endisset
```

- **@empty**: directive checks if a variable is **empty** (Not set, Null, "", [], 0).

- **@empty** is true for:

- null
 - empty string
 - empty array
 - 0

```
@empty($records)
    No records were found.
@endempty
```

- Use **@empty** when you want to show a message for **missing** or **empty data**.

Blade control statements and loops

- **Switch Statements**

- Switch statements can be constructed using the `@switch`, `@case`, `@break`, `@default` and `@endswitch` directives

- Use `@switch` when one value has many possible cases.

- *Best use cases:*

- *User role labels*
- *Status labels*
- *Category display*
- *Badge styles*

```
@switch($role)
    @case('admin')
        <span class="badge bg-danger">Admin</span>
        @break

    @case('editor')
        <span class="badge bg-info">Editor</span>
        @break

    @default
        <span class="badge bg-secondary">User</span>
@endswitch
```

- *If the logic becomes too long, move it to the controller.*

Blade control statements and loops

- Blade Loop Directives

- Blade provides loop directives that are similar to PHP.

- @for:

```
@for ($i = 0; $i < 10; $i++)  
    The current value is {{ $i }}  
@endfor
```

- @foreach:

```
@foreach ($users as $user)  
    <p>{{ $user->name }}</p>  
@endforeach
```

- @forelse:

```
@forelse ($users as $user)  
    <li>{{ $user->name }}</li>  
@empty  
    <p>No users found.</p>  
@endforelse
```

- @while:

```
@while ($condition)  
    <p>Looping...</p>  
@endwhile
```

Blade control statements and loops

- Blade Loop Directives

- Choosing the Right Loop

- Use `@for` when you know the number of repetitions.
 - Use `@foreach` when you want to display every item in a collection.
 - Use `@forelse` when the collection might be empty.
 - Use `@while` only when a condition controls the loop.

- Recommended for lists:

- It handles both the list and the empty message in one structure.

```
@forelse ($contacts as $contact)
    <tr>
        <td>{{ $contact['name'] }}</td>
    </tr>
@empty
    <tr>
        <td>No contacts found.</td>
    </tr>
@endforelse
```


Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- **Controlling loops and loop variables**
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Controlling loops and loop variables

- Controlling Loops

- When using loops, you may also skip the current iteration or end the loop using the `@continue` and `@break` directives:

- Use `@continue` to skip the current item.

```
@foreach ($users as $user)
    @continue($user->type == 1)

    <li>{{ $user->name }}</li>
@endforeach
```

- Use `@break` to stop the loop.

```
@foreach ($users as $user)
    <li>{{ $user->name }}</li>

    @break($user->number == 5)
@endforeach
```

- **Loop control** should be used carefully. If the logic becomes complicated, filter the data before sending it to the view.

Controlling loops and loop variables

- The **\$loop** Variable

- Inside Blade loops, Laravel provides a **\$loop** variable.
 - The **\$loop variable** also contains a variety of other useful properties
 - Useful properties:

<code>\$loop->index</code>	Current index, starting at 0
<code>\$loop->iteration</code>	Current iteration, starting at 1
<code>\$loop->remaining</code>	Number of items remaining
<code>\$loop->count</code>	Total number of items
<code>\$loop->first</code>	True if this is the first item
<code>\$loop->last</code>	True if this is the last item
<code>\$loop->even</code>	True if the iteration is even
<code>\$loop->odd</code>	True if the iteration is odd

Controlling loops and loop variables

- The **\$loop** Variable

- Inside Blade loops, Laravel provides a **\$loop** variable.
 - The **\$loop** variable also contains a variety of other useful properties
 - Example:

```
<tr @if($loop->even) class="table-primary" @endif>
  <th>{{ $loop->iteration }}</th>
  <td>{{ $contact['name'] }}</td>
</tr>
```

- *Use **\$loop** when you need numbering, row styling or first and last item checks.*

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- **Rendering views for collection**
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Rendering Views for Collection

- The `@each` directive renders one partial view for each item in a collection.

- Syntax:

```
@each('view_name', $collection, 'item', 'empty_view')
```

- `view_name` is the partial view.
- `$collection` is the data list.
- `item` is the variable name inside the partial.
- `empty_view` is displayed when the collection has no data.

- Example:

```
@each('contacts._row', $contacts, 'contact', 'contacts._empty')
```

- This keeps the main view shorter and moves each row into a separate partial.

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- **Styling the empty view**
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

Styling the Empty View

- An empty view is shown when there is no data to display.
 - Example — Create this file: *resources/views/contacts/_empty.blade.php*

```
<tr>
  <td colspan="7" class="text-center alert alert-danger">
    No contact found.
  </td>
</tr>
```

- *The user understands what happened.*
 - *the table does not look broken.*
 - *The interface feels complete.*
- *Good empty messages are short, clear and helpful.*

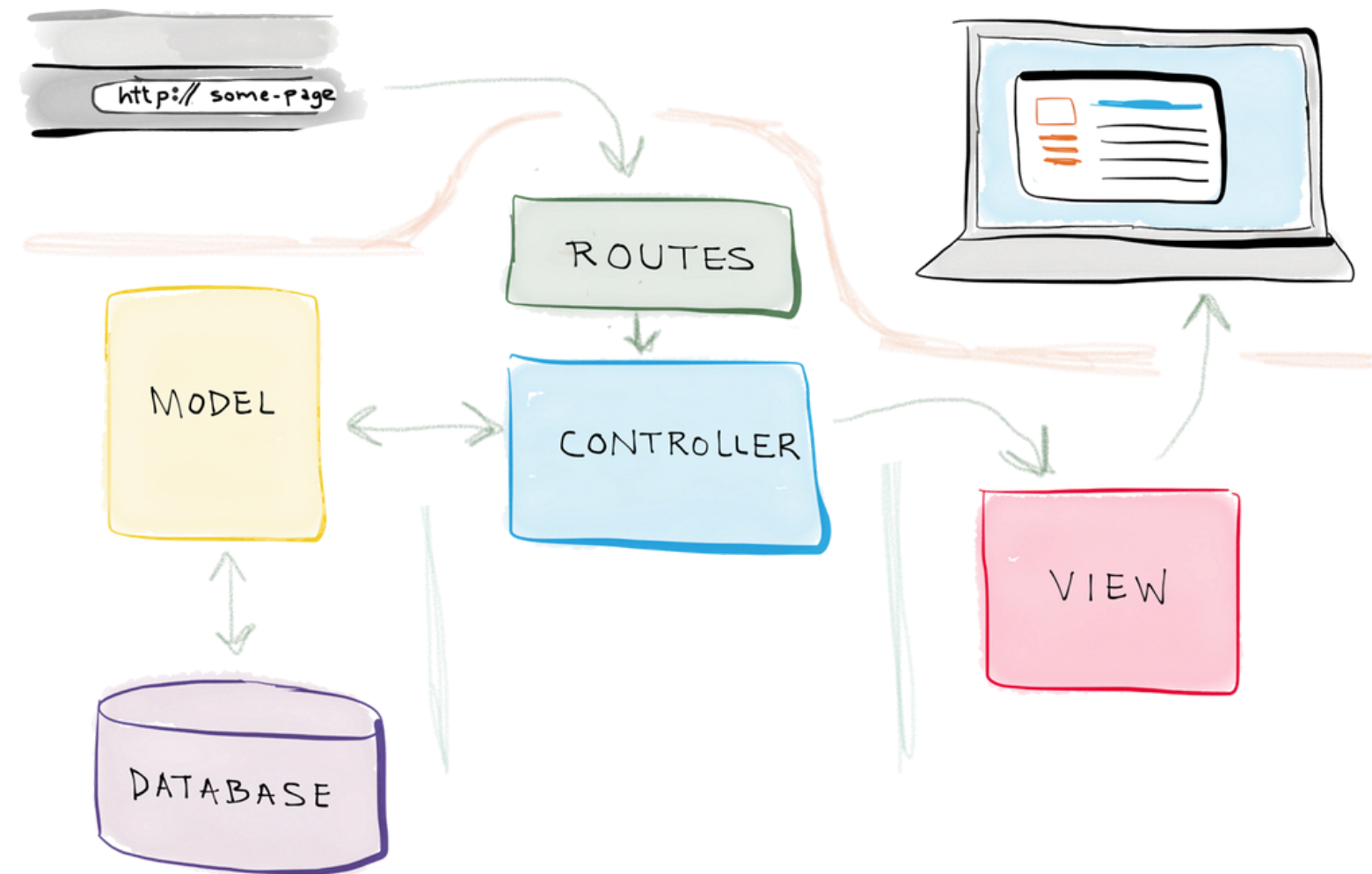
Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- **What is a Controller?**
- Routes and Controllers
- Creating Controllers
- Defining Routes for Controllers

What is a Controller?

- Laravel uses the **MVC** pattern.
 - **Model**: Handles data and database-related logic.
 - **View**: Displays the user interface using Blade templates.
 - **Controller**: Receives requests, manages logic and chooses the response.

- The **controller** connects
 - the **request**,
 - the **data**,
 - the **view**.



What is a Controller?

- A **controller** is a **PHP** class that manages application logic.
 - It acts as a middle layer between:
 - Routes
 - Models
 - Views
 - Example: A ContactController can manage all contact-related pages.
 - It may contain methods such as:
 - index()
 - show()
 - create()
 - store()
 - edit()
 - update()
 - destroy()
 - *Each method handles one action in the feature.*

What is a Controller?

- **Why Use Controllers?**

- Controllers make Laravel applications easier to organize.

- 1. Separation of Concerns**

- Keeps logic separate from routes and views.
- Routes handle URL mapping. Controllers handle logic. Views handle display.

- 2. Reusability**

- The same controller method can be reused or called from different places.
- Code can be reused across multiple routes.

- 3. Organization:**

- Related logic is grouped inside one class.
- Makes the application more structured and easier to maintain.

- 4. Maintainability**

- Large route files become difficult to read. Controllers keep the route file clean.
- *If the route function becomes too long, move the logic to a controller.*

What is a Controller?

- **Laravel Project Structure**

- Important Laravel folders:

- `routes/web.php` — Stores routes for web pages.
 - `routes/api.php` — Stores routes for API endpoints.
 - `app/Http/Controllers` — Stores controller classes.
 - `app/Models` — Stores model classes.
 - `resources/views` — Stores Blade view files.

- Example:

```
routes/web.php  
app/Http/Controllers/ContactController.php  
resources/views/contacts/index.blade.php
```

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- **Routes and Controllers**
- Creating Controllers
- Defining Routes for Controllers

Routes and Controllers

- Routes vs Controllers

Aspect	Routes	Controllers
Definition	Define the URL structure and link it to specific logic or views.	Encapsulate the application logic and manage requests for specific routes.
Purpose	Focus on mapping URLs to actions.	Focus on organizing and managing application logic.
Code Location	<code>routes/web.php</code> or <code>routes/api.php</code>	<code>app/Http/Controllers</code>
Complexity	Suitable for simple, single-action logic.	Better for grouping related actions and handling complex logic.
Reusability	Limited—logic is tied directly to the route.	High—logic can be reused across multiple routes or actions.

Routes and Controllers

- **Routes vs Controllers**

- **Routes:** For small applications or quick, simple actions.
- **Controllers:** For larger applications, complex logic, or reusable code.

- Example:

```
Route::get('/contacts', [ContactController::class, 'index']);
```

- This route maps `/contacts` to the `index` method of `ContactController`.

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- **Creating Controllers**
- Defining Routes for Controllers

Routes and Controllers

- **Creating a Controller**

- You can create a new controller by running command in terminal:

- Use Artisan to create a controller:

```
php artisan make:controller
```

- Example:

```
php artisan make:controller ContactController
```

- Laravel creates:

```
app/Http/Controllers/ContactController.php
```

directory

- By default, the new controller will extend the base Controller class that comes with Laravel

Routes and Controllers

- **Creating a Controller**

- Basic controller structure:

```
namespace App\Http\Controllers;

class ContactController extends Controller
{
    //
}
```

- Naming convention:

- Use a meaningful name followed by Controller.

- Examples:

- ContactController
 - CompanyController
 - ProductController

Routes and Controllers

- Basic Controller Example

- Example:

```
use App\Services\Data;

class ContactController extends Controller
{
    public function index()
    {
        $data = Data::$contacts_data;
        $companies = Data::$companies_data;

        return view('contacts.index', compact(
            'data',
            'companies'
        ));
    }
}
```

What this method does:

1. *Gets contact data.*
2. *Gets company data.*
3. *Sends both variables to the **contacts.index** view.*

*The method name **index** usually means display a list of resources.*

Outline

- Passing Data to Views
- Building Application Layout
- Blade Template Inheritance
- Including Sub-views
- Blade control statements and loops
- Controlling loops and loop variables
- Rendering views for collection
- Styling the empty view
- What is a Controller?
- Routes and Controllers
- Creating Controllers
- **Defining Routes for Controllers**

Defining Routes for Controllers

- After creating a controller, define a route that points to a controller method.

○ Example:

```
use App\Http\Controllers\ContactController;  
  
Route::get('/contacts', [ContactController::class, 'index']);
```

- `/contacts` is the URL.
- `ContactController::class` is the controller.
- `index` is the method that will run.
- This makes the route file cleaner because the logic is inside the controller.

Defining Routes for Controllers

- **Resource Controllers**

- A resource controller is used for **CRUD** features.

- **CRUD** means:

- Create
- Read
- Update
- Delete

- Create a resource controller:

```
php artisan make:controller ContactController --resource
```

- Register a resource route:

```
Route::resource('contacts', ContactController::class);
```

- Laravel automatically maps common URLs to controller methods.

Defining Routes for Controllers

- **API Controllers**

- **API controllers are used for API requests.**
 - They usually return JSON data instead of Blade views.

index	Display a list of resources
create	Show a form to create a resource
store	Save a new resource
show	Display one resource
edit	Show a form to edit a resource
update	Update an existing resource
destroy	Delete a resource

GET	/contacts	index
GET	/contacts/create	create
POST	/contacts	store
GET	/contacts/{id}	show
GET	/contacts/{id}/edit	edit
PUT/PATCH	/contacts/{id}	update
DELETE	/contacts/{id}	destroy

- Resource routes save time and follow Laravel conventions.

Defining Routes for Controllers

- **API Controllers**
 - **API controllers are used for API requests.**
 - They usually return JSON data instead of Blade views.
 - Create an API controller:

```
php artisan make:controller CompanyController --api
```

- Register an API resource route:

```
Route::apiResource('companies', CompanyController::class);
```

Defining Routes for Controllers

- **API Controllers**
 - **API controllers are used for API requests.**
 - Common API actions:
 - index
 - store
 - show
 - update
 - destroy
 - Why no **create** and **edit**?
 - APIs usually do not display HTML forms.

Defining Routes for Controllers

- **Choosing Route Types**

- **Single action route:**

```
Route::get('/users', [ContactController::class, 'index']);
```

- Use when one URL needs one method.

- **Resource route:**

```
Route::resource('contacts', ContactController::class);
```

- Use for full web CRUD pages.

- **API resource route:**

```
Route::apiResource('companies', CompanyController::class);
```

- Use for JSON CRUD endpoints.



THANK YOU

For your attention !

